Introduction au Machine learning

TP2 – Apprentissage Supervisé

Partie 1 : Préparation et Exploration

1. Importation des bibliothèques

Commencez par importer les bibliothèques nécessaires à l'analyse de données et au Machine Learning.

Python

```python
# Importation des librairies standards
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# Importation des modules Scikit-Learn
from sklearn import datasets
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

2. Chargement des données

Nous utiliserons le dataset **Iris**, qui contient des mesures pour 150 fleurs réparties en 3 espèces.

Consigne : Chargez le dataset et affichez les dimensions des variables X (caractéristiques) et Y (cible).

Python

```python
# Chargement du dataset
iris = datasets.load_iris()
X = iris.data
y = iris.target


# Affichage des informations
print(f"Dimensions de X : {X.shape}")
print(f"Noms des classes cibles : {iris.target_names}")
```

3. Séparation des données (Train / Test)

Pour évaluer correctement nos modèles, nous devons séparer les données.

Consigne : Divisez les données en un ensemble d'entraînement (**80%**) et un ensemble de test (**20%**). Fixez le random_state=42.

Python

```
# À COMPLÉTER
X_train, X_test, y_train, y_test = ...
```

Partie 2 : Classification (Comparaison des Modèles)

Vous allez entraîner trois modèles différents sur les mêmes données d'entraînement.

Modèle A : K-Nearest Neighbors (KNN)

L'algorithme KNN classe un point en fonction de la majorité de ses voisins.

Consigne :

1. Importez KNeighborsClassifier.
2. Entraînez le modèle avec **K=3** voisins.
3. Affichez l'Accuracy (précision globale).

Python

```
from sklearn.neighbors import KNeighborsClassifier

# À COMPLÉTER
knn = ...
knn.fit(X_train, y_train)
y_pred_knn = knn.predict(X_test)
print(f"Accuracy KNN : {accuracy_score(y_test, y_pred_knn):.2f}")
```

Modèle B : Arbre de Décision

L'arbre de décision utilise une suite de règles logiques.

Consigne : Entraînez un arbre avec une profondeur maximale de 3 (max_depth=3).

Python

```
from sklearn.tree import DecisionTreeClassifier
# À COMPLÉTER
tree = ...
tree.fit(..., ...)
y_pred_tree = ...
print(f"Accuracy Arbre : {accuracy_score(y_test, y_pred_tree):.2f}")
```

Modèle C : SVM (Support Vector Machine)

Le SVM cherche l'hyperplan qui sépare le mieux les classes.

Consigne : Entraînez un SVM avec un noyau linéaire (kernel='linear').

Python

```
from sklearn.svm import SVC
# À COMPLÉTER
svm = ...
svm.fit(..., ...)
y_pred_svm = ...
print(f"Accuracy SVM : {accuracy_score(y_test, y_pred_svm):.2f}")
```

Partie 3 : Évaluation Avancée

L'accuracy simple peut être trompeuse, surtout si les classes sont déséquilibrées. Analysons la performance du modèle SVM en détail.

1. Matrice de Confusion

Affichez la matrice de confusion pour visualiser exactement où le modèle SVM commet des erreurs (quelles classes sont confondues).

Python

```
# Création de la matrice
cm = confusion_matrix(y_test, y_pred_svm)

# Visualisation avec Seaborn
plt.figure(figsize=(6,4))
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.ylabel('Vraie classe')
plt.xlabel('Classe prédite')
plt.title('Matrice de Confusion (SVM)')
plt.show()
```

2. Rapport de Classification (Précision, Rappel, F1-Score)

Pour aller plus loin que l'accuracy, nous utilisons trois métriques clés pour chaque classe :

- **Precision :** La pureté des prédictions positives.
- **Recall (Rappel) :** La capacité à détecter toutes les instances d'une classe.
- **F1-Score :** La moyenne harmonique entre Précision et Rappel.

Consigne : Utilisez la fonction classification_report pour afficher ces métriques.

Python

```
# Import de la fonction nécessaire (si ce n'est pas déjà fait en haut)
from sklearn.metrics import classification_report
# Affichage du rapport complet
print("--- Rapport de Classification (SVM) ---")
print(classification_report(y_test, y_pred_svm, target_names=iris.target_names))
```

3. Validation Croisée (Cross-Validation)

Le score de test dépend de la manière dont on a coupé les données. Pour avoir une mesure plus robuste, utilisez la validation croisée.

Consigne : Calculez le score moyen du SVM sur 5 plis (folds).

Python

```
# Utilisation de cross_val_score
scores = cross_val_score(svm, X, y, cv=5)
print(f"Scores des 5 itérations : {scores}")
print(f"Accuracy Moyenne : {scores.mean():.2f}")
print(f"Écart-type : {scores.std():.2f}")
```

Partie 4 : Régression Linéaire (Cas Réel)

Nous allons prédire le **salaire** d'un employé en fonction de ses **années d'expérience**.

- **Dataset :** Salary_Data.csv (disponible via URL).

Consigne :

1. Chargez les données.
2. Entraînez une Régression Linéaire.
3. Visualisez le nuage de points et la droite de prédiction.

Python

```
from sklearn.linear_model import LinearRegression
# 1. Chargement des données
url = "https://raw.githubusercontent.com/sudarshan-koirala/Salary-Prediciton-based-on-Years-of-Experience/master/Salary_Data.csv"
df_salary = pd.read_csv(url)
X_sal = df_salary[['YearsExperience']] # Les Features (2D)
y_sal = df_salary['Salary']            # La Target (1D)
# 2. Entraînement
# À COMPLÉTER : Instanciez et entraînez le modèle 'reg'
reg = ...
reg.fit(..., ...)
# 3. Visualisation
plt.scatter(X_sal, y_sal, color='blue', label='Données réelles')
plt.plot(X_sal, reg.predict(X_sal), color='red', label='Droite de régression')
plt.xlabel("Années d'expérience")
plt.ylabel("Salaire")
plt.legend()
plt.show()
```

Question : Utilisez votre modèle pour prédire le salaire exact d'une personne ayant **12 ans** d'expérience.